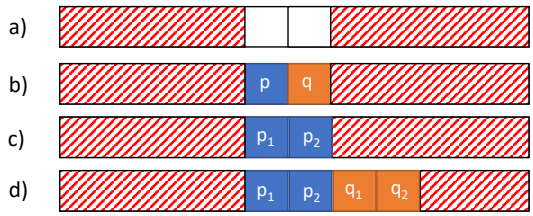

Fig. 3. Memory configuration: (a) almost full with only two bytes left, (b) after allocating p and q in (a), (c) after allocating p with twin allocation semantics in (a), (d) alternative configuration where twin allocation had enough space for both objects.

Our solution is to change allocation functions to reserve (at least) *two* blocks instead of a single one, as it happens in run-time implementations. We call this technique *twin allocation*, and we use it to formalize the notion that a program cannot guess the address of an object.

We will informally explain the concept of twin allocation with the following example:

```
char *p = malloc(1);
char *q = malloc(1);

*q = 0;
int v = (int)p + 1;  // equal to q?
*(char*)v = 1;

print(*q); // prints 0 or 1?
```

Since the address of `q` was not observed, we would like the compiler to be able to conclude that the program can only print `0`. However, as we have seen previously, if the memory is full, observing the address of one object may implicitly disclose the address of another object. In Figure 3(a) we show a possible memory configuration before our allocations, and (b) shows the configuration after allocating `p` and `q`.

With twin allocation, each allocation function reserves at least two blocks. Non-deterministically, one of the blocks is used and its address is returned, and the remaining blocks are marked as unreachable (i.e., it is UB to access those memory regions). By reserving two blocks, we guarantee there is enough non-determinism left such that the program cannot guess the address of a block even if the memory is full.

In Figure 3(c), we show that with twin allocation our previous example would simply run out of memory, and thus the program cannot continue and try to guess the address of `q`. In (d) we show another memory configuration where the space left was just enough to allocate two blocks for each allocation. Since we have two blocks per object, `malloc` can still return one of the two addresses non-deterministically, effectively inhibiting the program from guessing the address of an object. Even if a program is able to guess the address of, say, p_1 (and even p_2 as well), it is not able to guess which of the remaining addresses points to `q`: It cannot know which of q_1 and q_2 was used.

3.3 Summary

We have informally presented our memory model for LLVM IR. To support both high-level optimizations and low-level code, we split pointers into two categories. First, logical pointers, derived from allocation sites, for which we do data-flow dependence tracking. That is, a pointer `q` obtained by pointer arithmetic operations from `p` (e.g., `q = p + x`) can only access the same object as `p`. Second, physical pointers, derived from integer-to-pointer casts, for which we do not do data-flow